20

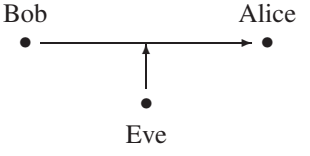
Application: Public key cryptography

This chapter presents one of the most interesting applications of the ideas from complexity theory and the algorithms from computer algebra: modern cryptography. After an introduction to the problem, we present six cryptographic algorithms: the famous RSA scheme, the Diffie-Hellman key exchange, two cryptosystems by ElGamal and by Rabin, and systems based on elliptic curves and short vectors in lattices.

It is satisfying to see how many of the computer algebra methods discussed in this text, certainly designed without this application in mind, have been useful for cryptography.

20.1. Cryptosystems

The scenario in this chapter is as follows. Bob wants to send a message to Alice in such a way that an eavesdropper Eve[1] listening to the transmission channel cannot understand the message. This is done by enciphering the message so that only Alice, possessing the right **key**, can decipher it, but Eve, having no access to the key, has no chance to recover the message.



The following are some of the ciphers that have been used in history.

- The **Caesar cipher**, which simply permutes the alphabet. The classical Caesar cipher used the cyclic shift by three letters $A \longmapsto D$, $B \longmapsto E$, $C \longmapsto F$, ..., $Y \longmapsto B$, $Z \longmapsto C$. For example, the word "CAESAR" is then enciphered as "FDHVDU". This cryptosystem is trivial to break: there are only 26 possibilities to try. More generally, one can use any of the $26! \approx 4 \cdot 10^{26}$ permutations

[1] Alice, Bob, and Eve are the leading characters of modern cryptography.

https://doi.org/10.1017/CBO9781139856065.025 Published online by Cambridge University Press

of a 26-letter alphabet. The key in this simple cryptosystem is the permutation; its inverse is used to decipher an encrypted message. However, if the eavesdropper knows in what language the original message was and thus knows the (approximate) probabilities for individual letters to occur in an average text, she may easily recover the message without knowledge of the key by performing a frequency analysis—provided the message is long enough. Thus this cipher is a convenient but highly insecure method.

- The **one-time pad**, which works as follows. To encipher a message of length n over the usual alphabet with 26 elements, a random sequence of n letters is chosen as a key and added letterwise modulo 26 to the message. The recipient then deciphers the encrypted message by subtracting the key letterwise. For example, if the message is "CAESAR", the key is "DOHXLG", and we identify the letters A,...,Z with the numbers $0, \ldots, 25$, then the ciphertext is "FOLPLX".

  The one-time pad is the only known provably secure cryptosystem (in an information theoretic sense), but has the disadvantage that the keys are of the same length as the message (keys cannot be reused without loss of security). It is an inconvenient but highly secure method.

In a more practical variant, a **pseudorandom number generator** is used to generate keys. Both Alice and Bob then use the same kind of generator parametrized by a **seed**, so that the two generators produce the same sequence when the same seed is used. Popular examples are the linear congruential generators.

However, we have seen in Section 17.2 that linear congruential generators are badly suited for cryptographical purposes since they are vulnerable to so-called "short vector" attacks. Today there are other types of pseudorandom number generators where no such attacks are known. In fact, several of the cryptosystems we are about to describe can be modified to work as pseudorandom generators. The variant of the one-time pad using pseudorandom number generators is no longer provably secure; its security heavily depends on the hardness of determining the elements of the pseudorandom sequence without knowledge of the seed.

In World War II, the German Wehrmacht[2] and Marine[3] used an enciphering machine called ENIGMA. This was a mechanical device where the keys were on cylinders; the underlying cipher was somewhat similar to the Caesar cipher but vastly more complicated. Its breaking by a British intelligence group at Bletchley Park under the famous computer scientist Alan Turing was vital for the Allied victory in the North Atlantic submarine war.

The main customers for cryptosystems used to be the military and secret services. Today, these systems are employed for all kinds of secure electronic data

[2] army
[3] navy

https://doi.org/10.1017/CBO9781139856065.025 Published online by Cambridge University Press

processing and communication like passwords, point-of-sale registers, banking machines, electronic cash, and internet commerce.

All of the classical cryptosystems are **symmetric** in the sense that the same key is used for both enciphering and deciphering, or at least the decryption key is easy (which means in polynomial time) to infer from the encryption key. A problem with this approach is that the number of keys grows quadratically with the number of parties if each party needs to communicate with each other party.

In classical cryptography, there never was a clear mathematical understanding of what "difficult to break" means; one could only take it to mean "the tricks that I know are not sufficient to break it". The security of a cryptosystem depends on the eavesdropper's cryptanalytic skills and her knowledge about the system. In the age of Caesar, it may have been reasonable to assume that the cryptanalyst has only a limited amount of computing power and that the encryption method may be kept secret from her. In the 20th century however, designers of cryptosystems, both classical and public key, have to take into account that potential eavesdroppers have high mathematical intelligence, access to supercomputers, complete knowledge about the encryption method except the keys, or can even channel in arbitrary parts of plaintext so that they have many plaintext/ciphertext pairs encrypted with the current key at their disposal.

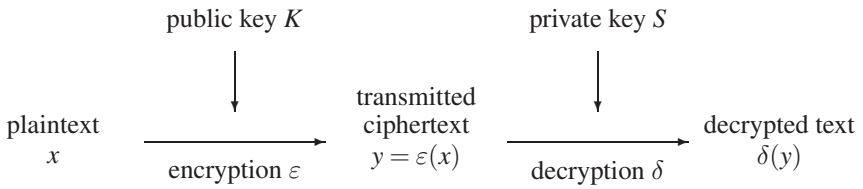


FIGURE 20.1: A public key cryptosystem.

Diffie & Hellman (1976) made a revolutionary proposal which has since then been known as **public key cryptography**. The idea is to have two different keys K and S for encryption and decryption, respectively, such that both encryption and decryption are "easy", but decryption without knowledge of S is "hard". Here "easy" means polynomial time, preferably almost linear or quadratic time in the message length. Figure 20.1 illustrates the situation. The name "public key cryptography" comes from the fact that the encryption key K may be publicly available. Since we want $x = \delta(y) = \delta(\varepsilon(x))$, δ is an inverse of ε. A function that is "easy" but its inverse is "hard" to compute without additional knowledge, like the encryption function in a public key cryptosystem, is called a **trapdoor function**. The keys K and S are called **public key** and **private key**, respectively. With such an **asymmetric cryptosystem**, n public-private key pairs are sufficient to permit secure communication among any two of n parties.

https://doi.org/10.1017/CBO9781139856065.025 Published online by Cambridge University Press

A cryptosystem is certainly broken when the private key is easy to find, but an appropriate notion of breaking a code is much more generous: a system is considered broken if there exists a Boolean predicate $B(x)$—say, the parity of x if x is an integer—and a polynomial-time probabilistic algorithm which takes $y = \varepsilon(x)$ as input and has a slightly better capability of predicting $B(x)$ than a random guess. Otherwise, the system is **semantically secure**; this is only possible for probabilistic encryption schemes, and the precise definition is a bit tricky.

There are several possibilities to make precise what "hard" means. Here is a list of some, ordered in increasing desirability.

- The inventor of the cryptosystem does not know of any polynomial time algorithm.
- Nobody knows of a polynomial time algorithm.
- Whoever breaks the system will probably in turn have solved a well-studied "hard" problem.
- Whoever breaks the system has in turn solved a well-studied "hard" problem.
- Whoever breaks the system has in turn solved an $\mathcal{NP}$–complete problem (Section 25.8).
- There is provably no (probabilistic) polynomial-time algorithm, as we have stipulated above.

At present, nobody knows of a cryptosystem fulfilling any of the last three requirements. However, it was a major conceptual breakthrough of the Diffie & Hellman proposal that the hitherto elusive notion of a "hard-to-break cipher" should be studied within the well-established framework of computational complexity.

Some of the modern proposals for cryptosystems have already been broken. Merkle & Hellman (1978) suggested a cryptosystem based on the **subset sum problem**. This system and several variants were broken using a basis reduction algorithm (Section 17.1). Another cipher proposed by Cade in 1985 (see Cade 1987) was based on the assumed hardness of the functional decomposition problem for polynomials: Given a polynomial f over a field F of degree n, decide if there exist polynomials $g, h \in F[x]$ of degree at least 2 such that $f = g \circ h = g(h)$, and if so, compute such g and h. The system was broken by Kozen & Landau (1989), who gave an algorithm for the problem with running time $O(n^3)$. Beyond our fairly simple scenario, modern cryptography studies many other tasks: electronic signatures and message authentication, multi-party communication, electronic cash, etc.

We now present some modern public key cryptosystems.

20.2. The RSA cryptosystem

We have already introduced in Section 1.2 this famous system, which is named after its inventors Rivest, Shamir & Adleman (1978) and is based on the assumed

https://doi.org/10.1017/CBO9781139856065.025 Published online by Cambridge University Press

hardness of factoring integers. The idea is that Alice randomly chooses two large (say 150-digit) primes $p \neq q$, and sets $N = pq$. Anybody who can factor N can break the system; ideally, we would also like the converse to be true, since numbers N with more than about 160 decimal digits seem out of the range of current integer factorization software (see the Notes and Chapter 19). Messages are encoded as sequences of elements of $\mathbb{Z}_N = \{0, \dots, N-1\}$. If, for example, we use the standard alphabet $\Sigma = \{A, \dots, Z\}$ of cardinality $\#\Sigma = 26$, then messages of up to $212 = \lfloor \log_{26} 10^{300} \rfloor$ letters can be uniquely represented by a single element of $\mathbb{Z}_N$, using the 26-adic representation. For example, the message "CAESAR" is encoded as

$$2 \cdot 26^0 + 0 \cdot 26^1 + 4 \cdot 26^2 + 18 \cdot 26^3 + 0 \cdot 26^4 + 17 \cdot 26^5 = 202\,302\,466 \in \mathbb{Z}_N.$$

If Alice wants to receive messages from Bob, she chooses $e \in \{2, \dots, \varphi(N) - 2\}$ with $\gcd(e, \varphi(N)) = 1$ at random, where φ is Euler's totient function (Section 4.2) and $\varphi(N) = \#\mathbb{Z}_N^\times = (p-1)(q-1)$. (She can also fix e, say $e = 3$.) Then she computes $d \in \{2, \dots, \varphi(N) - 2\}$ with $de \equiv 1 \bmod \varphi(N)$, using the Extended Euclidean Algorithm (Theorem 4.1), publishes the pair $K = (N, e)$ as her public key, and keeps her private key $S = (N, d)$ as well as p, q secret (the latter may even be discarded). The encryption and decryption functions $\varepsilon, \delta \colon \mathbb{Z}_N^\times \longrightarrow \mathbb{Z}_N^\times$ are defined by $\varepsilon(x) = x^e$ and $\delta(y) = y^d$. To send a message $x \in \mathbb{Z}_N^\times$ to Alice, Bob looks up her public key, computes $y = \varepsilon(x)$, and sends this to Alice, who computes $\delta(y)$, using her private key. Then, with $u \in \mathbb{Z}$ such that $de - 1 = u \cdot \varphi(N)$, we have

$$(\delta \circ \varepsilon)(x) = \delta(x^e) = x^{de} = x^{1 + u \cdot \varphi(N)} = x(x^{\varphi(N)})^u \equiv x \bmod N,$$

since $x^{\varphi(N)} = 1$ by Euler's theorem (Section 18.1). Although the latter is only valid if $\gcd(x, N) = 1$, actually $(\delta \circ \varepsilon)(x) = x$ is true for all x (Exercise 20.5). However, values of x that are not coprime to N lead to the factorization of N and thus to a break of the cryptosystem. Fortunately, if p and q are large and we assume that many or all messages x are likely to occur, then this will practically never happen.

We recall that a **polynomial-time reduction** from one problem X to another one Y is a polynomial-time algorithm for X making calls to a subroutine for Y. If polynomial-time reductions exist in both directions, then X and Y are **polynomial-time equivalent** (see Section 25.8). The following theorem is proven in Exercise 20.6.

THEOREM 20.1.
The following three problems are polynomial-time equivalent:

(i) factoring N,

(ii) computing $\varphi(N)$,

(iii) computing $d \in \mathbb{N}$ with $de \equiv 1 \bmod \varphi(N)$ from $K = (N, e)$.

https://doi.org/10.1017/CBO9781139856065.025 Published online by Cambridge University Press

Unfortunately, the theorem does *not* say that breaking the system means that one can factor integers efficiently, since there might be a successful attack that does not compute the private key at all.

The RSA scheme can also be used for **authentication**, where the sender of a message has to prove that he actually is the originator. This is also called a **digital signature**. If Bob wants to send a signed message x to Alice, he computes $y = \delta(x)$ using his own private key, and sends this to Alice, who looks up Bob's public key and recovers $x = \varepsilon(y)$. Since only Bob is assumed to know his private key, no forger would have been able to produce y, and Alice is convinced that the message originated from Bob. Instead of the whole message x, Bob might just sign a short digest of x obtained with a cryptographic hash function.

The authentication scheme may even be used in conjunction with the encryption scheme to ensure privacy. If ε_A, δ_A and ε_B, δ_B are Alice's and Bob's encryption and decryption functions, respectively, and Bob wants to send a signed message x to Alice that no one else can decipher, he computes $y = \varepsilon_A(\delta_B(x))$, and sends this to Alice, who first decrypts $\delta_A(y) = \delta_B(x)$ and then $x = \varepsilon_B(\delta_A(y))$, at the same time assuring herself that the message originates from Bob.

20.3. The Diffie–Hellman key exchange protocol

Diffie & Hellman (1976) presented this scheme for two parties to agree upon a common key for future communication with a symmetric cryptosystem rather than a public key cryptosystem. An example might be the seed of a pseudorandom generator to be used for a one-time pad. Let $q \in \mathbb{N}$ be a "large" prime power (say with about 1000 bits) and g be a generator of $\mathbb{F}_q^\times$, the multiplicative group of the finite field $\mathbb{F}_q$ (Section 25.4). Then $\mathbb{F}_q^\times$ is isomorphic to the cyclic (additive) group $\mathbb{Z}_{q-1}$, via $g^i \longleftrightarrow i$. The protocol works as follows.

1. Alice and Bob agree on q and g, which may be public.
2. Alice secretly chooses $a \in \mathbb{Z}_{q-1}$ at random, computes $u = g^a \in \mathbb{F}_q^\times$, and sends u to Bob.
3. Bob secretly chooses $b \in \mathbb{Z}_{q-1}$ at random, computes $v = g^b \in \mathbb{F}_q^\times$, and sends v to Alice.
4. Alice computes $g^{ab} = v^a$.
5. Bob computes $g^{ab} = u^b$.

Both parties may use g^{ab} as a common key for further communication with a symmetric cryptosystem. In this context, the following problems play a central role.

https://doi.org/10.1017/CBO9781139856065.025 Published online by Cambridge University Press

PROBLEM 20.2. *(Diffie-Hellman Problem, DH) Given $g^a, g^b \in \mathbb{F}_q^\times$, compute g^{ab}.*

PROBLEM 20.3. *(Discrete Logarithm Problem, DL) Given $g^a \in \mathbb{F}_q^\times$, compute a.*

It is conjectured that DL is a "hard" problem. The fastest algorithms for DL have the same subexponential running times as for factoring integers, and DL seems unlikely to be $\mathcal{NP}$-complete. An eavesdropper knowing q, g and the transmitted u, v but not a or b has to solve DH to find out g^{ab}, which in turn is polynomial-time reducible to DL. It is, however, not clear whether DL is polynomial-time reducible to DH.

The currently best estimate for the computation of discrete logarithms in $\mathbb{F}_q^\times$ is $\exp(O(\sqrt[3]{n \log^2 n}))$ word operations, where $n \approx \log_2 q$ is the binary length of a description for an element of $\mathbb{F}_q$ (Gordon 1993). A cryptosystem using $q = 2^{1013}$ was marketed, and $\exp(\sqrt[3]{1013 \log_2^2 1013}) > 1.6 \cdot 10^{20}$; a machine performing one calculation per nanosecond takes about 500 years to achieve this number.

20.4. The ElGamal cryptosystem

As before, $\mathbb{F}_q^\times$ is a "large" finite field with q elements, and g is a generator of $\mathbb{F}_q^\times$. To receive messages from Bob, Alice randomly chooses $S = b \in \mathbb{Z}_{q-1}$ as her private key and publishes $K = (q, g, g^b)$. If Bob wants to send a message x to Alice, he looks up her public key, randomly chooses $k \in \mathbb{Z}_{q-1}$, computes g^k and xg^{kb}, and sends $y = (u, v) = (g^k, xg^{kb})$ to Alice, who computes the original message as $x = v/u^b$. Computing x from y without knowing S is polynomial-time equivalent to the Diffie-Hellman Problem.

A practical problem in implementing the Diffie-Hellman scheme or the ElGamal system is that exponentiation in $\mathbb{F}_q^\times$ is theoretically easy ($O(n^3)$ word operations for $q = 2^n$ using classical arithmetic), but not fast enough to achieve high throughput. One can, however, achieve time $O^\sim(n^2)$.

20.5. Rabin's cryptosystem

This is based on the hardness of computing square roots modulo $N = pq$, where p, q are two "large" primes as in the RSA scheme. The factorization of N can be reduced to the computation of square roots, as follows. Choose some $x \in \mathbb{Z}_N$ and compute $y = \sqrt{x^2}$. Then $x^2 \equiv y^2 \bmod N$, or equivalently, $pq = N \mid (x+y)(x-y)$. If $x \not\equiv \pm y \bmod N$, then this gives us the factorization of N. This is the main idea for Dixon's random squares algorithm in Section 19.5.

Thus to send a message x to Alice, Bob uses her public key N, and sends $y \equiv x^2 \bmod N$. Alice can compute the two square roots each of y modulo p and q, by equal-degree factorization (Section 14.3), and combine them via the Chinese

https://doi.org/10.1017/CBO9781139856065.025 Published online by Cambridge University Press

Remainder Algorithm. There are various tricks to deal with the choice among the four different answers computed by Alice.

However, the system's use as a signature scheme is vulnerable to an **active attack**: if Eve chooses a random x and gets Alice to sign a message $y \equiv x^2 \bmod N$, by returning a square root z of y modulo N, then with probability $1/2$, $\gcd(x-z,N)$ will be a proper factor of N. The system is not considered secure for this reason.

20.6. Elliptic curve systems

These work similarly to the ElGamal system, except that the multiplicative group $\mathbb{F}_q^\times$ is replaced by the additive group of an elliptic curve E over $\mathbb{F}_q$ (see Section 19.7). The curve is public, and so is a point P on E. Let n be the order of P in E. Alice chooses $a \in \{2,\dots,n-2\}$ at random and publishes aP. When Bob wants to send her the message $(m_1,m_2) \in \mathbb{F}_q^2$, with $m_1,m_2 \in \mathbb{F}_q$, he chooses a random k, computes $kP = (r_1,r_2)$ and $k \cdot aP = (s_1,s_2)$ in $\mathbb{F}_q^2$, and sends $(y_1,y_2,y_3,y_4) = (r_1,r_2,s_1m_1,s_2m_2) \in \mathbb{F}_q^4$ to Alice. She then computes $a \cdot (r_1,r_2) = a \cdot kP = (s_1,s_2)$ and retrieves Bob's message as $(m_1,m_2) = (s_1^{-1}y_3, s_2^{-1}y_4)$.

Notes. **20.1.** The one-time pad is in Vernam (1926). Cryptographically strong pseudo-random number generators, whose prediction is thought to be computationally hard, are discussed in Lagarias (1990). An algorithm with time $O^\sim(n)$ for functional decomposition is in von zur Gathen (1990a, 1990b); see Exercise 20.3.

20.2. The security of individual bits in the RSA scheme has been discussed by several researchers; see Näslund (1998) and Håstad & Näslund (1998) for references. It is generally hard to predict, and even harder to predict the future. Nevertheless, Odlyzko (1995b) extrapolates past progress and concludes that 1500 to 10000 bits are needed for a number (used in a cryptosystem) to be safe against factoring attempts. Attacks via basis reduction on small RSA exponents and other cryptographic applications are discussed in Nguyen & Stern (2001).

20.3. McCurley (1990) gives an overview on discrete logarithm algorithms. Maurer & Wolf (1999) reduce DL to DH in some special cases.

20.4. ElGamal (1985) gives his cryptosystem. Fast exponentiation in finite fields $\mathbb{F}_{2^n}$ can be achieved using Gauß periods, normal bases, and fast arithmetic (von zur Gathen & Nöcker 1997, Gao, von zur Gathen & Panario 1998, Gao, von zur Gathen, Panario & Shoup 2000).

20.6. Public key cryptosystems based on elliptic curves were invented by Miller (1986) and Koblitz (1987b). Menezes (1993) and Blake, Seroussi & Smart (1999) present comprehensive treatments.

Exercises.

20.1 As in Section 20.1, we identify the letters A, B, C, …, Z with the elements $0,1,2,\dots,25$ of $\mathbb{Z}_{26}$.

(i) The word "OAYBGFQD" is the result of an encryption with a Caesar cipher, which maps each letter $x \in \mathbb{Z}_{26}$ to $x+k$, where $k \in \mathbb{Z}_{26}$ is the key. What are the cleartext and the key?

https://doi.org/10.1017/CBO9781139856065.025 Published online by Cambridge University Press

(ii) The word "MLSELVY" is the ciphertext after encryption with the one-time pad using the key "IAMAKEY". Find the cleartext.

20.2$^{\longrightarrow}$ This exercise is about a variant of the following password encryption scheme suggested by Purdy (1974), before the advent of public-key cryptography (Diffie & Hellman 1976). Let $p = 2^{64} - 59$, which is the largest prime below our processor's assumed word length 2^{64}, encode a 13-letter password w^* over the 26-letter alphabet $\{\mathrm{A}, \mathrm{B}, \ldots, \mathrm{Z}\}$ as a number w using 26-adic notation, and consider $w \in \mathbb{F}_p$. This makes sense, since $26^{13} < p$. Then w is encrypted as $f(w) \in \mathbb{F}_p$, where

$$f = x^{2^{24}+17} + a_1 x^{2^{24}+3} + a_2 x^3 + a_3 x^2 + a_4 x + a_5 \in \mathbb{F}_p[x],$$

for some specific values $a_1, \ldots, a_5 \in \mathbb{F}_p$. The pairs $(\text{login-name}, f(\text{password}))$ are stored in a public file. When a user logs on and types in her password w^*, $f(w)$ is calculated and checked against the entry in the file.

(i) Let $a_1 = 2$, $a_2 = 37$, $a_3 = -42$, $a_4 = 15$, $a_5 = 7$, and $w^* = \text{RUMPELSTILTZK}$. Calculate w and $f(w)$.

(ii) How many arithmetic operations in $\mathbb{F}_p$ are used to calculate $f(w)$ from w?

(iii) Let $v \in \mathbb{F}_p$. The core of the algorithm in Exercise 14.20, which calculates $\{w \in \mathbb{F}_p : f(w) = v\}$, is the computation of $x^p \text{ rem } f$. Extrapolating the timings from Figure 9.10, you may assume that one multiplication modulo f can be done in about one hour. Since f is sparse, the reduction modulo f is inexpensive. How long does the computation of $x^p \text{ rem } f$ take approximately? What do you conclude about the security of this system (on today's computers)?

20.3* (Kozen & Landau 1989, von zur Gathen 1990a) Let F be a field and $f \in F[x]$ of degree n. A **functional decomposition** of f is given by two polynomials $g, h \in F[x]$ of degrees at least two such that $f = g \circ h = g(h)$. If no such decomposition exists, then f is **indecomposable**. Obviously a necessary condition for the existence of a decomposition is that n be composite.

(i) Let $f = g \circ h$ be a functional decomposition and $c, d \in F$ with $c \neq 0$. Show that $f = g(cx+d) \circ (h-d)/c$ is also a functional decomposition. Find a functional decomposition $f/\operatorname{lc}(f) = g^* \circ h^*$ into monic polynomials $g^*, h^* \in F[x]$, with the same degrees as g, h, and such that $h(0) = 0$. We call such a decomposition **normal**.

(ii) Let $f = g \circ h$ be a normal decomposition, $r = \deg g$, $s = \deg h$, and $f^* = \operatorname{rev}(f) = x^n f(x^{-1})$ and $h^* = \operatorname{rev}(h) = x^s h(x^{-1})$ the reversals of f and h, respectively. Prove that $f^* \equiv (h^*)^r \bmod x^s$.

(iii) Let $f = g_1 \circ h_1$ be another normal decomposition with $r = \deg g_1$ and $s = \deg h_1$ and assume that r is coprime to $\operatorname{char} F$. Prove that $h = h_1$ and $g = g_1$. Hint: Uniqueness of Newton iteration (Theorem 9.27).

(iv) Consider the following algorithm, which works even over rings.

ALGORITHM 20.4 Functional decomposition of polynomials.

Input: A monic polynomial $f \in R[x]$ of degree $n > 3$ and a nontrivial divisor r of n, where R is a ring (commutative, with 1) of characteristic coprime to r.

Output: Either a normal decomposition $f = g \circ h$ with $g, h \in R[x]$ and $\deg g = r$, or "no such decomposition".

1. $f^* \longleftarrow \operatorname{rev}(f)$, $\quad s \longleftarrow n/r$
   { compute rth root of f^* via Newton iteration }
   call the Newton iteration algorithm 9.22 to compute $h^* \in R[x]$ of degree less than s with $h^*(0) = 1$ and $(h^*)^r \equiv f^* \bmod x^s$
   $h \longleftarrow x^s h^*(x^{-1})$
2. **call** Algorithm 9.14 to compute the h-adic expansion $f = h^r + g_{r-1}h^{r-1} + \cdots + g_1 h + g_0$ of f, with $g_{r-1}, \ldots, g_0 \in R[x]$ of degrees less than s
3. **if** $g_i \in R$ for all i **then return** $g = x^r + \sum_{0 \le i < r} g_i x^i$ and h
   else return "no such decomposition"

https://doi.org/10.1017/CBO9781139856065.025 Published online by Cambridge University Press

Prove that the algorithm works correctly, and show that it takes $O(\mathsf{M}(n)\log r)$ additions and multiplications in R. What goes wrong if $\gcd(r, \operatorname{char} R) > 1$?

(v) Apply the algorithm to find a decomposition of $f = x^6 + x^5 + 2x^4 + 3x^3 + 3x^2 + x + 1 \in \mathbb{F}_5[x]$.

20.4 Let $N = 8051 = 97 \cdot 83$.

(i) The public key in a RSA cryptosystem is $K = (N, e) = (8051, 3149)$. Find the corresponding private key $S = (N, d)$.

(ii) A message x has been encrypted using K, and the resulting ciphertext is 694. What is x?

20.5 Let $p, q \in \mathbb{N}$ be distinct primes, $N = pq$, $K = (N, e)$ the public key, and $S = (N, d)$ the private key in a RSA cryptosystem, such that $d, e \in \mathbb{N}$ satisfy $de \equiv 1 \bmod \varphi(N)$.

(i) In Section 20.2, we have assumed that messages x to be encrypted are coprime to N. Prove that the RSA scheme also works if this condition is violated. Hint: Chinese Remainder Theorem.

(ii) Show that the intruder Eve, who has intercepted the ciphertext $\varepsilon(x)$ but does not know the private key S, can easily break the system if x is not coprime to N.

20.6* In this exercise, you are to prove Theorem 20.1. So let $N = pq$ for two distinct primes $p, q \in \mathbb{N}$.

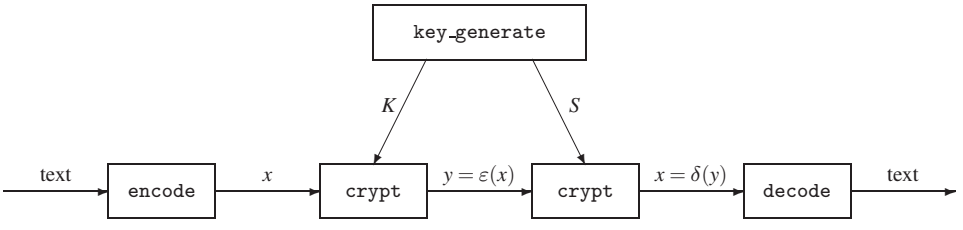
(i) Show how to compute p, q from the knowledge of N and $\varphi(N)$. Hint: Consider the quadratic polynomial $(x-p)(x-q) \in \mathbb{Z}[x]$.

(ii) Suppose that you are given a black box which on input $e \in \mathbb{N}$ decides whether it is coprime to $\varphi(N)$, and if so, returns $d \in \{1, \ldots, \varphi(N)-1\}$ such that $de \equiv 1 \bmod \varphi(N)$. Give an algorithm using this black box which computes $\varphi(N)$ in time $(\log N)^{O(1)}$. Hint: Find a "small" e coprime to $\varphi(N)$.

20.7⟶ (i) Program a procedure `key_generate` that generates a pair (K, S) of keys for the RSA cryptosystem, such that $K = (N, e)$ is the public key, $S = (N, d)$ is the private key, N is the product of two random 100 bit prime numbers, $e \in \{2, \ldots \varphi(N) - 2\}$ is chosen uniformly at random, and $d \in \{2, \ldots, \varphi(N) - 2\}$ satisfies $de \equiv 1 \bmod \varphi(N)$.

(ii) Design a coding for short strings of English words with at most 30 letters, including punctuation marks, parentheses, and blanks, as integers between 0 and $N-1$, and write corresponding procedures `encode` and `decode`.

(iii) Write a procedure `crypt` for encrypting and decrypting with the RSA cryptosystem. Its arguments should be a number in $\mathbb{Z}_N$ and a key.



(iv) Check your programs with sample messages of your choice, and produce some timings.

Research problems.

20.8 Reduce (in probabilistic polynomial time) factoring integers to breaking RSA (or some other cryptosystem).

20.9 Reduce DL to DH in polynomial time.

https://doi.org/10.1017/CBO9781139856065.025 Published online by Cambridge University Press

https://doi.org/10.1017/CBO9781139856065.025 Published online by Cambridge University Press

https://doi.org/10.1017/CBO9781139856065.025 Published online by Cambridge University Press